LUMEN: Linear Attention with a Gated Delta Rule

Mira Okonkwo, Tarun Vaidya, Elsa Lindqvist (Halvard Institute)

Abstract

Linear attention replaces the softmax with a kernel feature map so that a model keeps a fixed-size state instead of a growing key-value cache. Its weakness is recall: the state only ever accumulates, so old associations interfere with new ones. We present LUMEN, a linear attention layer whose state is updated by a gated delta rule. A scalar gate decays the state and a delta step overwrites the value stored under the current key. LUMEN trains in parallel over chunks, runs as a recurrence at inference, and matches a Transformer of the same size in perplexity while improving associative recall.

1 Introduction

Softmax attention compares every query with every key, so its cost grows quadratically, as O(n^2) in the sequence length n, and its key-value cache grows linearly with the context. Linear attention removes both costs by writing each key-value pair into a matrix-valued state and reading it with the query. The price is that the state has a fixed capacity and no way to remove what it stored, which shows up as poor recall on long inputs.

Two remedies have been studied separately. Decay gates shrink the whole state at every step, which frees capacity but forgets uniformly. The delta rule instead replaces the value stored under a key, which edits memory precisely but never frees space. LUMEN combines the two in one update and shows that the combination is better than either alone.

Our contributions are threefold. First, we define the gated delta rule and show that it is a single step of online gradient descent on a regression loss with weight decay. Second, we derive a chunk-wise parallel form that trains efficiently on accelerators. Third, we train models up to 1.3B parameters and report language modelling and recall results.

2 Background: Linear Attention

A causal linear attention layer keeps a state S_t, a matrix with one row per value dimension and one column per key dimension. At each step it adds the outer product of the value and the key and reads the output with the query:

$$S_t = S_{t-1} + v_t k_t^\top, \quad o_t = S_t q_t \tag{1}$$

This recurrence has constant memory per token. Because nothing is ever removed, the state saturates once the number of stored pairs exceeds the key dimension, and retrieval of an early key returns a mixture of everything written since.

3 Method

3.1 The Gated Delta Rule

The delta rule writes the difference between the new value and the value currently retrieved for the key, scaled by a learned writing strength beta_t. We add a gate alpha_t in (0, 1), computed from the input token, that multiplies the state before the write:

$$S_t = \alpha_t S_{t-1} (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top \tag{2}$$

In this update the gate alpha_t decays the whole state before the delta update writes the new association, so the layer can clear its memory at a document boundary while still editing single associations inside a document. With alpha_t fixed at 1 the update reduces to the plain delta rule, and with beta_t fixed at 0 it reduces to a decaying state.

Equation (2) is one step of online gradient descent with learning rate beta_t on the loss ||S k_t - v_t||^2, followed by weight decay with factor alpha_t. Keys are L2-normalised so that the update is a contraction and training stays stable.

3.2 Chunk-wise Parallel Form

Running Equation (2) token by token is sequential and slow on accelerators. We split the sequence into chunks and compute each chunk's contribution with matrix multiplications, passing only the state from one chunk to the next. The products of the delta terms inside a chunk are kept in a compact WY representation, so no d by d matrix is formed per token. In all experiments we use a chunk size of 64 tokens, which balanced memory and speed on our hardware.

4 Experiments

We train models of 340M and 1.3B parameters on 100B tokens of the synthetic WebMix corpus with the same data order for every architecture. We report perplexity on WikiText, accuracy on LAMBADA, and accuracy on the MQAR associative recall task with 256 key-value pairs.

Table 2: Language modelling and recall at 1.3B parameters.

Model	Params	Wiki ppl	LAMBADA	MQAR
Transformer++	1.3B	16.85	51.9	88.4
Linear attention	1.3B	19.73	44.1	21.6
Delta rule	1.3B	17.40	49.8	70.2
LUMEN	1.3B	16.42	52.8	79.5

LUMEN improves WikiText perplexity over the Transformer++ baseline and closes most of the recall gap that plain linear attention leaves. The gate and the delta step are both needed: removing the gate costs 0.98 perplexity, and removing the delta step costs 58 points of MQAR accuracy.

Figure 2: Training throughput at 16K context. LUMEN processes 41K tokens per second per GPU, 2.3 times the FlashAttention-2 baseline, and its advantage grows with the context length.

5 Limitations

The fixed-size state remains a bottleneck for tasks that must reproduce long inputs verbatim: exact copying of long spans remains weaker than in softmax attention, and in our tests a hybrid with two softmax layers was needed to match it. We also have not studied models above 1.3B parameters.

References

[1] J. Halloran and P. Ibe. Fast weight memories for sequence learning. Journal of Neural Recurrence, 1993.

[2] K. Marr, S. Oduya and T. Brandt. Kernelized attention with feature maps. In Proceedings of the Learning Systems Conference, 2020.

[3] R. Castellan. The delta rule revisited for linear transformers. arXiv preprint, 2021.

[4] A. Petrov and L. Huang. Decay gates for recurrent attention. In Proceedings of the Sequence Modelling Workshop, 2023.